Question: I am doing the front end in Next js and another team member is doing the back end in Python, Django Rest framework and the database in Postgre SQL. Can we develop them separately and connect them through API-s when we are ready or are we dependent on each other during the development process?

Yes, you can develop the frontend and backend separately and connect them later via APIs, but some early coordination is important to avoid integration issues.

How to Work Independently & Connect Later

- Define the API contract early → Agree on how the frontend will request data and what responses it will get (e.g., REST API endpoints, request/response formats).
- Use mock data → While the backend is being developed, you can use JSON placeholders or tools like Mock Service Worker (MSW) to simulate API responses.
- Follow a modular approach → Keep your frontend API calls in a separate utility (e.g., `api.ts` file in Next.js), so you can easily switch from mock data to real API calls later.

Recommended Development Workflow

1 – Backend Team:

- Develop **Django REST Framework (DRF) API endpoints** and connect to PostgreSQL.
- Document API routes & response formats.

2 – Frontend (Your Part):

- Build UI components using **Next.js, React, Tailwind CSS, and ShadCN**.
- Use **mock API responses** to develop and test the UI.

3 – Integration Phase:

- Replace mock data with real API calls once the backend is ready.
- Test API connectivity and handle authentication, pagination, error handling, etc.

Show me a step by step guide for defining the API contract, using mock data and following a modular approach based on these two documents:

Frontend overview and general plan (1).pdf
European Shelter Dog Adoption Platform.pdf

Show me one step by step plan for the front end and a separate step by step plan for the back end

Based on the documents, here's a **step-by-step guide** for defining the API contract, using mock data, and following a modular approach.

Since you and your teammate are developing **the frontend (Next.js)** and **the backend (Django + DRF + PostgreSQL) separately**, this guide ensures smooth integration.

API contract definition (First Step for Both Teams)

Before developing independently, you must define how the **frontend** and **backend** communicate.

Step 1: Agree on API Endpoints & Data Structure

- **Decide what data the frontend needs** and how the backend will provide it.
- **List endpoints**, their request parameters, and expected responses.

Example API Contract for Dog Search & Profile

Endpoint	Method	Request Parameters	Response Example
`/api/dogs`	`GET`	`breed, age, size, country, sex` (optional filters)	`{ dogs: [{ id: 1, name: "Buddy", breed: "Labrador", age: 2, size: "Large", sex: "Male", shelter: { name: "Berlin Shelter", location: "Germany" } }] }`

`/api/dogs/{id}`	`GET`	`id` (dog's unique ID)	`{ id: 1, name: "Buddy", description: "Friendly Labrador", images: [], shelter: { name: "Berlin Shelter", contact: "shelter@email.com" } }`
`/api/shelters`	`GET`	None	`{ shelters: [{ id: 1, name: "Berlin Shelter", location: "Germany" }] }`

Use tools like Postman or Swagger for API documentation.

Step-by-Step Plan for Frontend (Next.js)

Phase 1: Setup API Handling in a Modular Way

1. Create API utility file (`lib/api.ts`)

- This keeps API calls separate from UI components.

```
export async function fetchDogs(filters = {}) {
  const query = new URLSearchParams(filters).toString();
  const res = await fetch(`/api/dogs?${query}`);
  return res.json();
}

export async function fetchDogById(id: number) {
  const res = await fetch(`/api/dogs/${id}`);
  return res.json();
}
```

2. Use API calls in React components

- Example: Fetching dogs for the search page.

```
import { useEffect, useState } from "react";
import { fetchDogs } from "@/lib/api";

export default function DogSearch() {
  const [dogs, setDogs] = useState([]);

  useEffect(() => {
    fetchDogs().then(data => setDogs(data.dogs));
  }, []);

  return (
    <div>
      {dogs.map(dog => (
        <p key={dog.id}>{dog.name} - {dog.breed}</p>
      ))}
    </div>
  );
}
```

Phase 2: Use Mock Data While Backend is in Progress

Since the backend might not be ready yet, use **Mock Service Worker (MSW) or static JSON data**.

1. Install MSW for API mocking

```
npm install msw --save-dev
```

2. Set up an API mock in `mocks/handlers.js`

```
import { rest } from "msw";

export const handlers = [
```

```
  rest.get("/api/dogs", (req, res, ctx) => {
    return res(ctx.json({
      dogs: [{ id: 1, name: "Buddy", breed: "Labrador", age: 2, size:
"Large", sex: "Male" }]
    }));
  }),
];
```

3. Enable mocking in `mocks/browser.js`

```
import { setupWorker } from "msw";
import { handlers } from "./handlers";

export const worker = setupWorker(...handlers);
```

4. Run MSW in `pages/_app.tsx`

```
import { useEffect } from "react";
import { worker } from "@/mocks/browser";

if (process.env.NODE_ENV === "development") {
  worker.start();
}
```

Now, your frontend works **without needing the real backend**.

Step-by-Step Plan for Backend (Django + DRF)

Phase 1: Define Models in Django

1. Set up Django models in `models.py`

```
from django.db import models

class Shelter(models.Model):
    name = models.CharField(max_length=255)
    location = models.CharField(max_length=255)
    email = models.EmailField()

class Dog(models.Model):
    name = models.CharField(max_length=255)
    breed = models.CharField(max_length=255)
    age = models.IntegerField()
    size = models.CharField(max_length=10, choices=[('Small',
'Small'), ('Medium', 'Medium'), ('Large', 'Large')])
    sex = models.CharField(max_length=6, choices=[('Male', 'Male'),
('Female', 'Female')])
    shelter = models.ForeignKey(Shelter, on_delete=models.CASCADE,
related_name="dogs")
```

Phase 2: Set Up API Endpoints in Django REST Framework

2. Create Serializers in `serializers.py`

```
from rest_framework import serializers
from .models import Dog

class DogSerializer(serializers.ModelSerializer):
    class Meta:
        model = Dog
        fields = '__all__'
```

3. Create API Views in `views.py`

```
from rest_framework import generics
from .models import Dog
```

```
from .serializers import DogSerializer

class DogList(generics.ListCreateAPIView):
    queryset = Dog.objects.all()
    serializer_class = DogSerializer

class DogDetail(generics.RetrieveAPIView):
    queryset = Dog.objects.all()
    serializer_class = DogSerializer
```

4. Set up API URLs in `urls.py`

```
from django.urls import path
from .views import DogList, DogDetail

urlpatterns = [
    path('dogs/', DogList.as_view(), name='dog-list'),
    path('dogs/<int:pk>/', DogDetail.as_view(), name='dog-detail'),
]
```

Phase 3: Test API Locally

5. Run Django Development Server

```
python manage.py runserver
```

6. Use Postman or `curl` to test the API

```
curl http://127.0.0.1:8000/dogs/
```

Now the backend is ready to integrate with the frontend.

Phase 4: Connect Frontend & Backend

When the backend is ready, update the API calls in **Next.js** to fetch real data.

1. Update `lib/api.ts` in the frontend

```
export async function fetchDogs(filters = {}) {
  const query = new URLSearchParams(filters).toString();
  const res = await
fetch(`https://your-backend-url.com/api/dogs?${query}`);
  return res.json();
}
```

2. Remove MSW (Mock Data) from the frontend

```
npm uninstall msw
```

Now, the frontend is **fully connected** to the backend!

Summary: Parallel Development with API Integration

- **Step 1: Define API Contract** → Agree on endpoints and responses.
- **Step 2: Frontend Uses Mock Data** → Develop UI without waiting for the backend.
- **Step 3: Backend Develops API** → Implement endpoints and test them separately.
- **Step 4: Integrate Frontend & Backend** → Replace mock API calls with real ones.

Documenting the API Contract using Swagger/OpenAPI

To make your **API contract clear and accessible**, you should document it using **Swagger (OpenAPI Specification)**. This will help both frontend and backend developers understand how to interact with the API.

Step 1: Install Django REST Framework and drf-yasg (Swagger Generator)

On the **backend side**, install the Swagger documentation tool:

```
pip install drf-yasg
```

Step 2: Configure Swagger in Django

In your Django app, update **`urls.py`** to include Swagger documentation:

```
from django.urls import path, re_path
from rest_framework import permissions
from drf_yasg.views import get_schema_view
from drf_yasg import openapi
from .views import DogList, DogDetail

schema_view = get_schema_view(
    openapi.Info(
        title="European Shelter Dog Adoption API",
        default_version="v1",
        description="API documentation for the European Shelter Dog
Adoption Platform",
        contact=openapi.Contact(email="support@dogadoption.com"),
        license=openapi.License(name="MIT License"),
    ),
    public=True,
    permission_classes=(permissions.AllowAny,),
)
```

```
urlpatterns = [
    path('dogs/', DogList.as_view(), name='dog-list'),
    path('dogs/<int:pk>/', DogDetail.as_view(), name='dog-detail'),

    # Swagger URLs
    re_path(r'^swagger/$', schema_view.with_ui('swagger',
cache_timeout=0), name='schema-swagger-ui'),
    re_path(r'^redoc/$', schema_view.with_ui('redoc',
cache_timeout=0), name='schema-redoc'),
]
```

Step 3: Test the Swagger API Docs

Run the Django server:

```
python manage.py runserver
```

Now, visit:

- **Swagger UI:** `http://127.0.0.1:8000/swagger/` (Interactive API docs)
- **ReDoc:** `http://127.0.0.1:8000/redoc/` (Alternative API docs format)

Step 4: Define API Models in Swagger

You can add **detailed documentation for each API endpoint** in your Django views.

Example: Document the `/dogs/` API Endpoint

Modify `views.py` to include **Swagger annotations**:

```
from rest_framework import generics
from .models import Dog
```

```
from .serializers import DogSerializer
from drf_yasg.utils import swagger_auto_schema
from drf_yasg import openapi

class DogList(generics.ListCreateAPIView):
    queryset = Dog.objects.all()
    serializer_class = DogSerializer

    @swagger_auto_schema(
        operation_description="Get a list of adoptable dogs",
        manual_parameters=[
            openapi.Parameter('breed', openapi.IN_QUERY,
description="Filter by breed", type=openapi.TYPE_STRING),
            openapi.Parameter('age', openapi.IN_QUERY,
description="Filter by age", type=openapi.TYPE_INTEGER),
            openapi.Parameter('size', openapi.IN_QUERY,
description="Filter by size (Small, Medium, Large)",
type=openapi.TYPE_STRING),
            openapi.Parameter('sex', openapi.IN_QUERY,
description="Filter by sex (Male, Female)",
type=openapi.TYPE_STRING),
        ],
        responses={200: DogSerializer(many=True)}
    )
    def get(self, request, *args, **kwargs):
        return self.list(request, *args, **kwargs)
```

Now, Swagger will automatically generate documentation for this endpoint.

Step 5: Testing the API in Swagger

- Open `http://127.0.0.1:8000/swagger/`
- Click on `/dogs/`
- Try sending requests directly from Swagger UI

Summary

- **Swagger UI** (`/swagger/`) → Interactive API documentation
- **ReDoc UI** (`/redoc/`) → Alternative API documentation
- **OpenAPI Specification** helps frontend and backend teams stay in sync
- **Test API endpoints directly from Swagger**

How should the frontend define and document the the API Contract using Swagger/OpenAPI?

Defining and Documenting the API Contract for the Frontend Using OpenAPI (Swagger)

While Swagger is typically used for backend API documentation, the **frontend team** can also use OpenAPI to:
– **Define expected API responses** (based on the backend contract).
– **Generate TypeScript types** for better type safety.
– **Mock API responses for testing.**

Step 1: Create an OpenAPI Specification File

Create a **JSON or YAML file** to define the API contract expected by the frontend.

Option 1: OpenAPI in JSON (`openapi.json`)

```
{
  "openapi": "3.0.0",
  "info": {
    "title": "European Shelter Dog Adoption API",
    "version": "1.0.0",
    "description": "API contract between frontend (Next.js) and
backend (Django DRF)."
  },
  "paths": {
    "/dogs": {
      "get": {
        "summary": "Fetch a list of adoptable dogs",
        "parameters": [
          { "name": "breed", "in": "query", "schema": { "type":
```

```
"string" } },
          { "name": "age", "in": "query", "schema": { "type":
"integer" } },
          { "name": "size", "in": "query", "schema": { "type":
"string" } },
          { "name": "sex", "in": "query", "schema": { "type":
"string" } }
        ],
        "responses": {
          "200": {
            "description": "List of dogs",
            "content": {
              "application/json": {
                "schema": {
                  "type": "object",
                  "properties": {
                    "dogs": {
                      "type": "array",
                      "items": {
                        "$ref": "#/components/schemas/Dog"
                      }
                    }
                  }
                }
              }
            }
          }
        }
      }
    },
    "components": {
      "schemas": {
        "Dog": {
          "type": "object",
          "properties": {
            "id": { "type": "integer" },
            "name": { "type": "string" },
            "breed": { "type": "string" },
```

```
          "age": { "type": "integer" },
          "size": { "type": "string" },
          "sex": { "type": "string" }
        }
      }
    }
  }
}
```

Step 2: Host the OpenAPI Spec for Frontend Reference

Store this file in a public location (e.g., **GitHub, Vercel, or SwaggerHub**) so it remains up to date.

- **SwaggerHub:** Upload it for free hosting (https://swagger.io/tools/swaggerhub).
- **GitHub Pages:** Store in a repo and access it as a raw JSON file.
- **Vercel (if hosting the frontend):** Store in `public/openapi.json` and access it at `/openapi.json`.

Step 3: Use OpenAPI to Generate TypeScript Types for API Responses

Instead of manually writing TypeScript interfaces, you can **auto-generate types from the OpenAPI spec**.

Install OpenAPI-to-TypeScript Converter

```
npm install openapi-typescript
```

Generate TypeScript Types

```
npx openapi-typescript https://your-domain.com/openapi.json -o
types/api.ts
```

This will create a `types/api.ts` file with:

```ts
export interface Dog {
  id: number;
  name: string;
  breed: string;
  age: number;
  size: string;
  sex: string;
}
```

Now, your Next.js API calls will be strongly typed.

Step 4: Use Type-Safe API Calls in Next.js

Now that we have TypeScript types, we can use them in API calls.

API Utility File (`lib/api.ts`)

```ts
import { Dog } from "@/types/api";

export async function fetchDogs(filters: Record<string, string | number> = {}): Promise<{ dogs: Dog[] }> {
  const query = new URLSearchParams(filters as Record<string, string>).toString();
  const res = await fetch(`/api/dogs?${query}`);
  if (!res.ok) throw new Error("Failed to fetch dogs");
  return res.json();
}
```

Example Usage in a Component

```ts
import { useEffect, useState } from "react";
import { fetchDogs } from "@/lib/api";
```

```tsx
import { Dog } from "@/types/api";

export default function DogList() {
  const [dogs, setDogs] = useState<Dog[]>([]);

  useEffect(() => {
    fetchDogs().then(data => setDogs(data.dogs));
  }, []);

  return (
    <div>
      {dogs.map(dog => (
        <p key={dog.id}>{dog.name} - {dog.breed}</p>
      ))}
    </div>
  );
}
```

Now, Next.js components are type-safe and fully documented!

Summary: API Contract for the Frontend

1. **Define API expectations** in **OpenAPI format (`openapi.json`)**.
2. **Host the OpenAPI file** on **SwaggerHub, GitHub, or Vercel** for reference.
3. **Generate TypeScript types** with `openapi-typescript`.
4. **Use TypeScript types** in API calls to prevent errors.
5. **Ensure consistent API requests** to match the backend.

So defining and documenting the API contract is mainly back ends task?

Yes, defining and documenting the **API contract** is typically the backend's responsibility, but it's a collaborative effort between the backend and frontend teams.

Here’s how the process generally works:

1. Backend's Role in API Contract

- **Define Endpoints**: The backend defines the routes (e.g., `GET /dogs`, `POST /dogs`, etc.), methods (GET, POST, PUT, DELETE), and request/response formats (JSON, status codes).
- **Create API Documentation**: The backend should document the API so the frontend team knows what data to expect and how to interact with it. This can be done using tools like **Swagger (OpenAPI)**, **Postman**, or **DRF's browsable API**.
    - For example, you can use **Swagger** to generate interactive API documentation that helps both teams see available endpoints and their expected inputs/outputs.

2. Frontend's Role in API Contract

- **Understand API Structure**: The frontend team should review the API documentation and ensure they understand the structure, expected data types, and authentication methods.
- **Give Feedback**: If the frontend team finds issues or wants changes (e.g., additional fields or endpoints), they can suggest improvements.

3. Collaborate on Changes

- As the project progresses, both teams will likely iterate on the API contract. If the frontend team finds that they need different or additional data, the backend team will adjust the API accordingly.

In short, the **backend is primarily responsible for defining and documenting the API contract**, but the frontend team must review and provide feedback to ensure the API supports all their needs.